

VL-JEPA

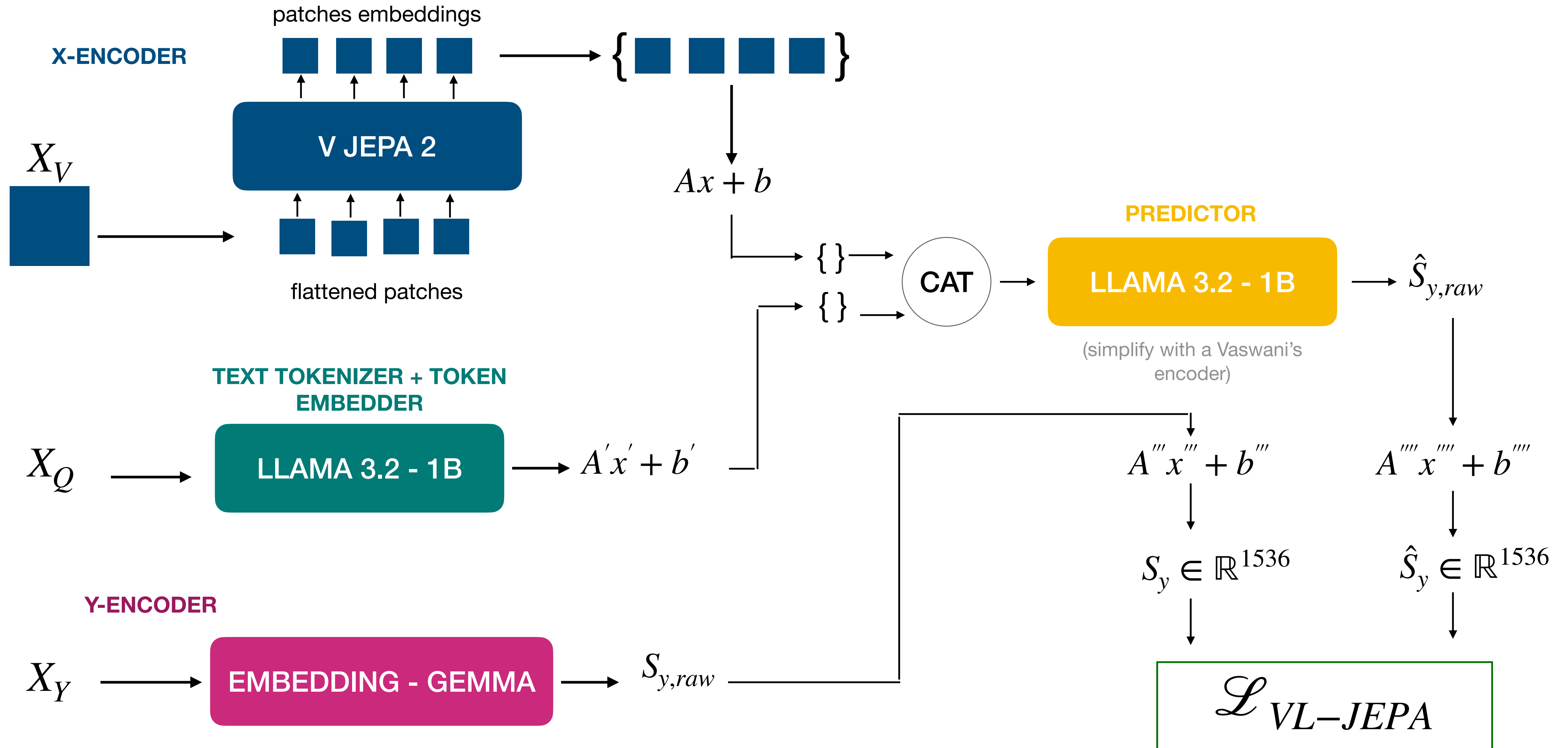
Joint Embedding Predictive Architecture for Vision Language [1]

Chen et al. - 2026

(Edoardo Canti - Argomento di Approfondimento - Deep Learning Applications 25/26)

1. Introduction

VL-JEPA Architecture



Loss Functions for JEPAs

In **JEPA** the training is guided via a **loss** in the **embedding space** $\mathcal{L}_{VL-JEPA} = D(\hat{S}_y, S_y)$

Several losses can be used, in their article authors uses **InfoNCE Loss**:

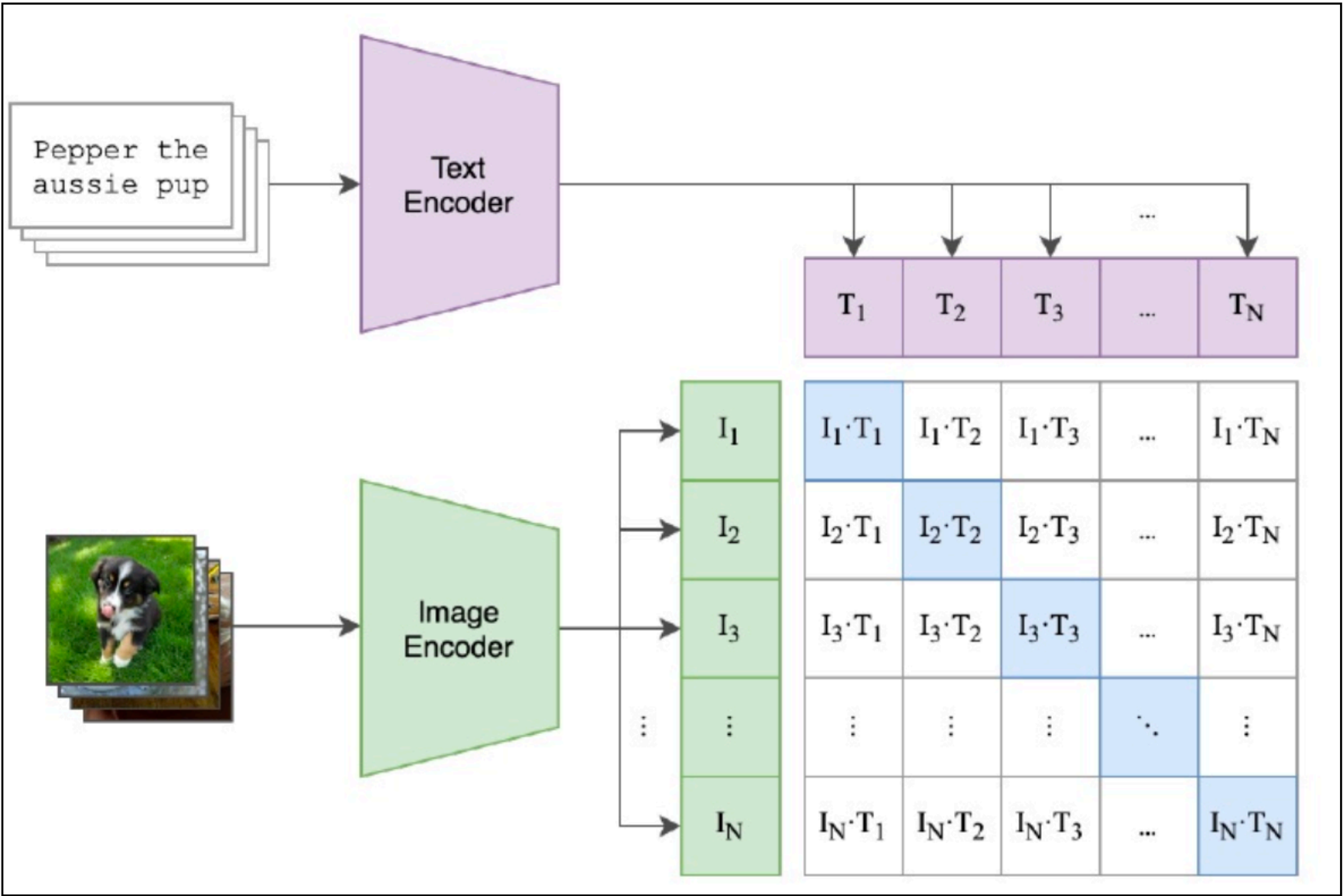
$$\mathcal{L}_{VL-JEPA} = -\frac{1}{2B} \sum_{i=1}^B \left(\log \frac{e^{\langle S_{y_i}, \hat{S}_{y_i} \rangle / \tau}}{\sum_{j=1}^B e^{\langle S_{y_i}, \hat{S}_{y_j} \rangle / \tau}} + \log \frac{e^{\langle \hat{S}_{y_i}, S_{y_i} \rangle / \tau}}{\sum_{j=1}^B e^{\langle \hat{S}_{y_i}, S_{y_j} \rangle / \tau}} \right)$$

eq1: InfoNCE Loss

Interpretation:

The domain of action is the embedding space. During the training the goal is to **reduce** the **distance** between the **learnt representation** of the input and the **learnt representation** of the target.

We already saw something similar...



Usage of InfoNCE in CLIP

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{2B} \sum_{i=1}^B \left(\log \frac{e^{\langle u_i, v_i \rangle / \tau}}{\sum_{j=1}^B e^{\langle u_i, v_j \rangle / \tau}} + \log \frac{e^{\langle v_i, u_i \rangle / \tau}}{\sum_{j=1}^B e^{\langle v_i, u_j \rangle / \tau}} \right)$$

eq2:General form of InfoNCE Loss

Model	u	v
CLIP	Image representation	Text representation
VL-JEPA	Predictor output representation (Visual + Textual Query)	Target text Representation

Why do we care about CLIP?

CLIP can be considered as the building block of VLM architectures.

VLM architectures, like LLMs, are auto-regressive models trained over (self-supervised) next token prediction tasks.

Imagine an Image Captioning task: the next token prediction training nature of VLMs, forces the model to strictly match the expected output (next token).

Authors claim: there are several ways to describe the same object, we don't need the model to perfectly align with the target.



What this image shows?

- A red super car
- An italian super car
- A document image of an email

Why VL-JEPA should be better?

By choosing to **NOT** penalize the model on exact **Next Token Prediction**:

1. The model learns how to **discard useless semantic information**
2. The model learns how to **represent more abstract similar concepts in same regions in the space**
3. The **learnt representation space is smooth** (we will see later World Predictions and Action Anticipations)
4. The **embedded representation** already **embodies the concept** and there is no need to rely on **high latency** autoregressive tasks for tokens predictions (authors assess that this let VL-JEPA a more adequate architecture for real time tasks like image captioning through smart glasses).

2. Architectural Details

X-Encoder (visual encoder)

X- Encoder is a **V JEPA 2**, which can be viewed as a three dims extension of I-JEPA *(please read the gray paragraph below)*

In I-JEPA an entire image is divided in patches (as in classic ViT), instead of being trained on classification tasks applying the attention mechanism on each patches it:

Let $\{P\}$ be the set of patches extracted from an image

1. An encoder encodes a subset of patches $\{C\}$ in $\{P\}$ (**contexts**).
2. Another encoder maps the **target** patches $\{T\} = \{P\} \setminus \{C\}$
3. The predictor (via attention mechanism) considers $\{C\}$ and the positional embeddings of elements in $\{T\}$ and **tries to learn the representations of them**.

Probably is too simple to say that V-JEPA extends I JEPA to video (a sequence of frames), but this is not in the focus of the presentation.

Personal claim: Ideally we could consider also the “simpler” ViT Transformer (An Image is Worth 16x16 Words) because the representations predictions is applied later.

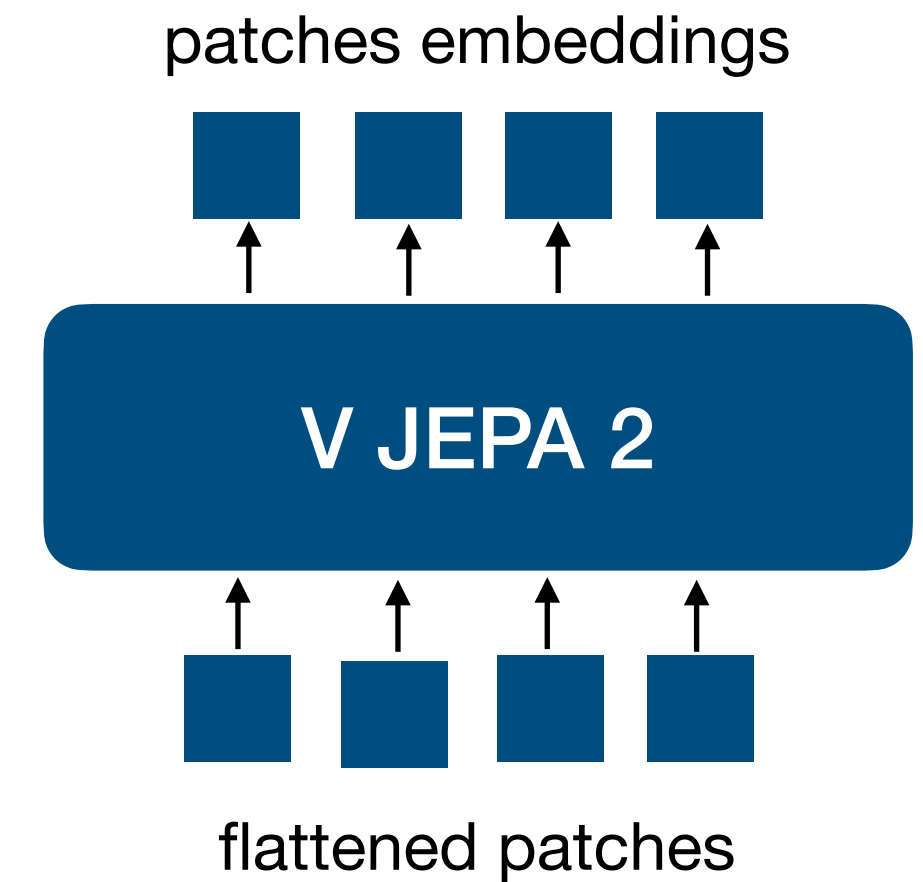


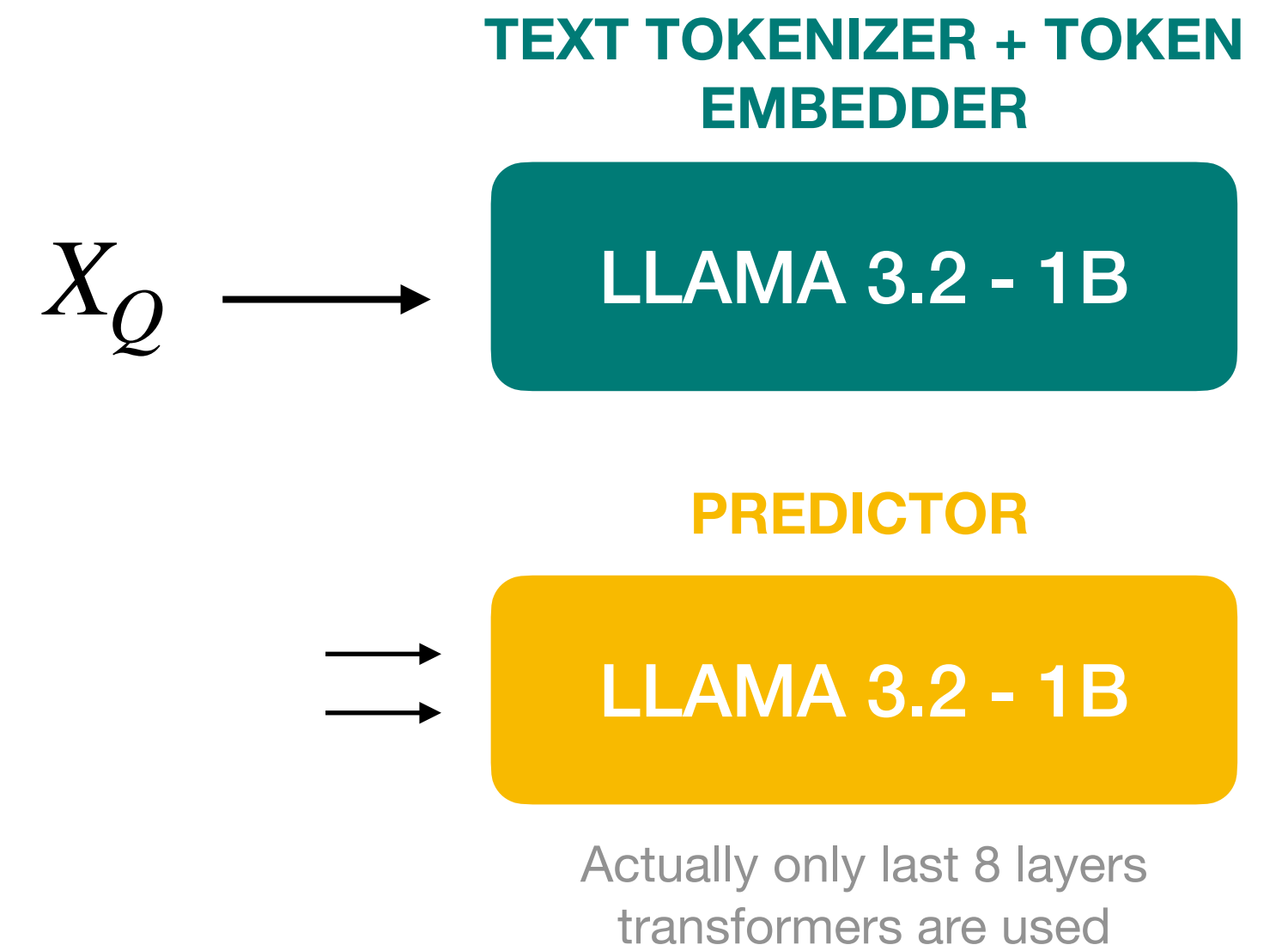
Image from I-JEPA paper

Predictor and Query Tokenizer/Embedder

Although **Predictor**, **Tokenizer** and **Embedder** are different components are collected here because of their internal architecture.

All of these comes from the **LLAMA-3.2 1B** family:

1. **Text Embedder** and **Tokenizer**: are used to preprocess the textual query.
2. The **predictor** itself is actually used to provide an embedding of the **concatenation** of **Visual Embeddings** and **Textual embeddings**



The model is available at 🙌:

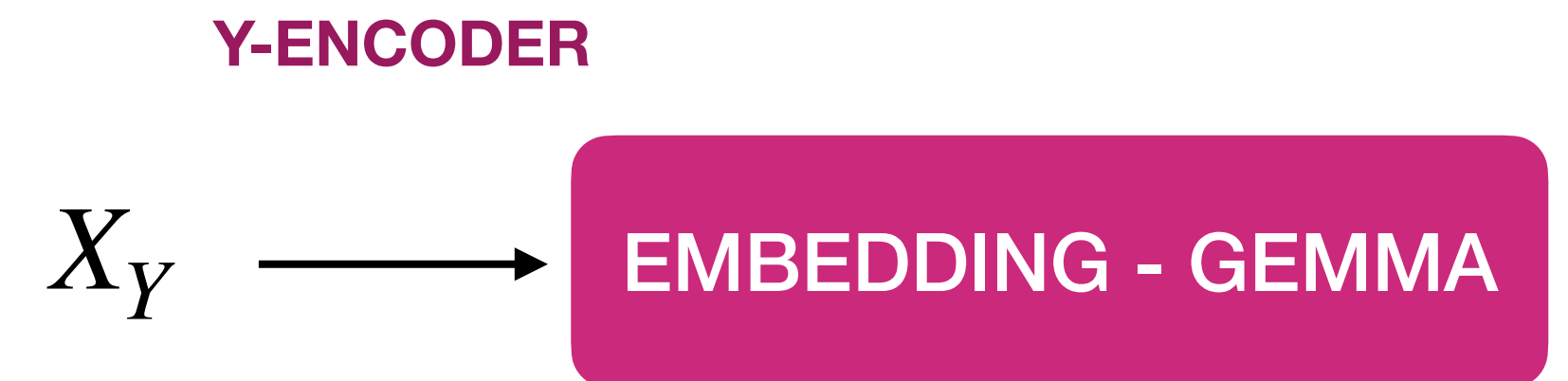
```
from transformers import AutoTokenizer, AutoModel
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.2-1B")
model = AutoModel.from_pretrained("meta-llama/Llama-3.2-1B")
```

Note: The huggingface repo is actually gated because the access is restricted (model card doesn't report the output shape of the last hidden layer for embedding), found "online" (LLM USED HERE) it is 2048

Y-Encoder (target text encoder)

Y-Encoder is used to map the **target text** to its **representation**

Please remind that a refined output of the Y-encoder (refined because its about will be than passed to a projection layer) will be used to compute the InfoNCE with the Predictor output.



Also this model is available at [🤗](#) and as we can read from the model card:

1. it accepts text string such as: questions, prompt or entire document to be embedded, with a maximum ctx length of 2048 tokens
2. outputs a **vector representation of the text with dimension 768** (or 512, 256, 128)

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("google/embeddinggemma-300m")
```

Note: The huggingface repo is actually gated because the access is restricted (model card report the embedding dimensions)

Linear projectors

Now it should be clear the reason of those **linear projectors**:

From **X encoder** to **Predictor** $Ax + b$

From **Text-Query Encoder** to **Predictor** $A'x' + b'$

From **Y Encoder** to **Embedding Space** $A''x'' + b''$

From **predictor** to **Embedding Space** $A'''x''' + b'''$

3. Training VL-JEPA

Two Stages Training 1/4

VL-JEPA is trained in **two different stages**:

1. Large-Scale Pretraining: *query-free* pretraining is used in order to **guarantee a robust vision-language alignment**.

Type of datasets: caption data

- **Image-Text** datasets: Datacomp, YFCC-100M
- **Video-Text** datasets: Action100M

This first part is itself divided into:

a) first 100k iterations sampling only from **Image-Text** datasets

Here the only information is provided by the **single image “semantic”** and the **text**

Internally the model is learning **“what the picture shows”**

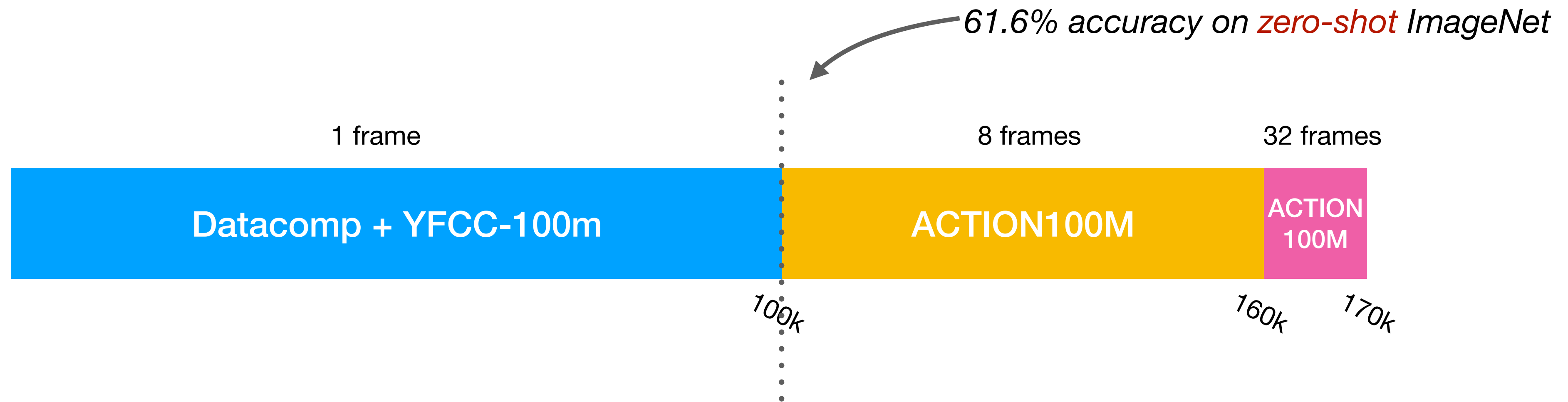
Since each image is 1 frame, this guarantees a **batch size** of **24k** (this a blessing for **InfoNCE**)

$$\mathcal{L}_{\text{VL-JEPA}} = -\frac{1}{2B} \sum_{i=1}^B \left(\log \frac{e^{\langle S_{y_i}, \hat{S}_{y_i} \rangle / \tau}}{\sum_{j=1}^B e^{\langle S_{y_i}, \hat{S}_{y_j} \rangle / \tau}} + \log \frac{e^{\langle \hat{S}_{y_i}, S_{y_i} \rangle / \tau}}{\sum_{j=1}^B e^{\langle \hat{S}_{y_i}, S_{y_j} \rangle / \tau}} \right)$$

Two Stages Training 2/4

b) After 100k iterations **Video-Text datasets** are used, starting from 8 frames per input, arriving then to 32 frames per input

Ideally in this phase the model should learn **what an action is** (after that it learnt how to interpret a single object)



The model resulting from the pretraining phase is named **VL-JEPA BASE**

The pretraining took 4 weeks, over a cluster of 24 nodes, each mounting 8 Nvidia H200...

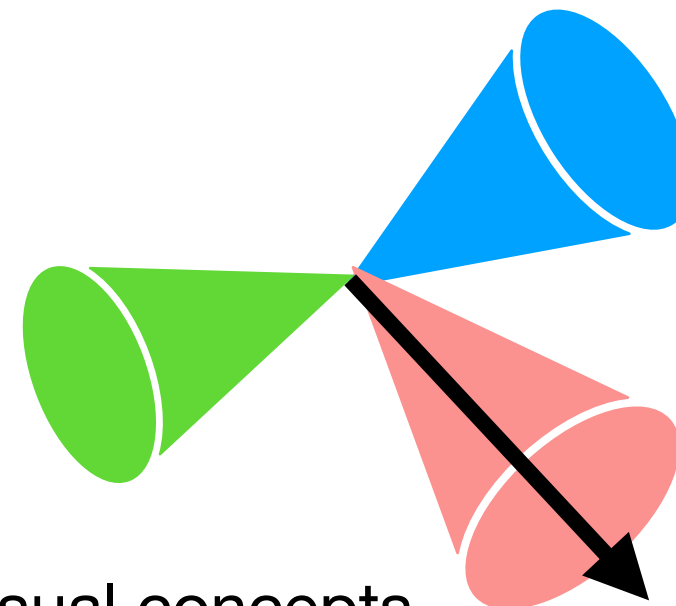
Two Stages Training 3/4

Large Scale Pretraining Summary



Linear Projectors are excluded for graphical reasons

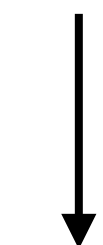
V JEPA 2



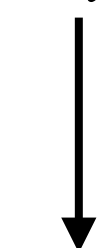
Space region of higher visual concepts
of fast objects (images of objects moving)

LLAMA 3.2 - 1B

$\hat{S}_{y,raw}$



$\hat{S}_y$

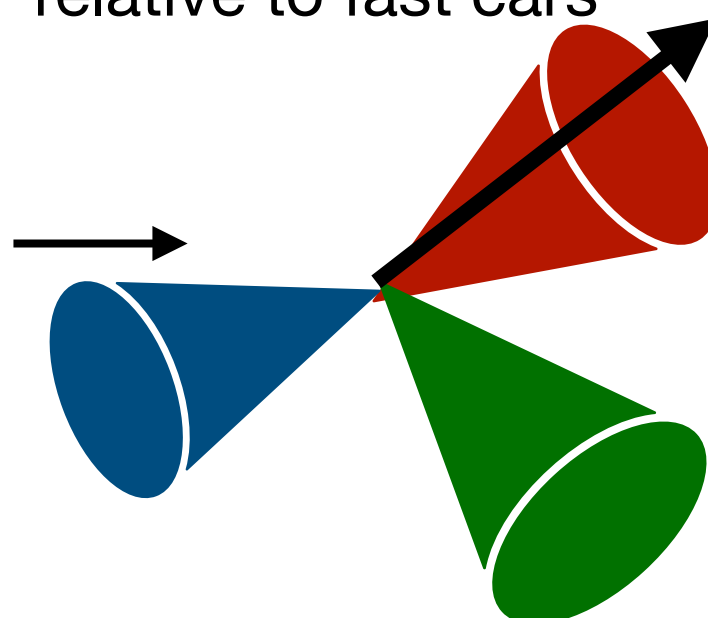


red super car



EMBEDDING - GEMMA

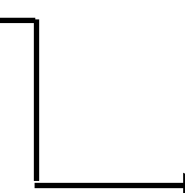
Y-ENCODER



Space region of higher textual concepts
relative to fast cars

$S_{y,raw}$

S_y



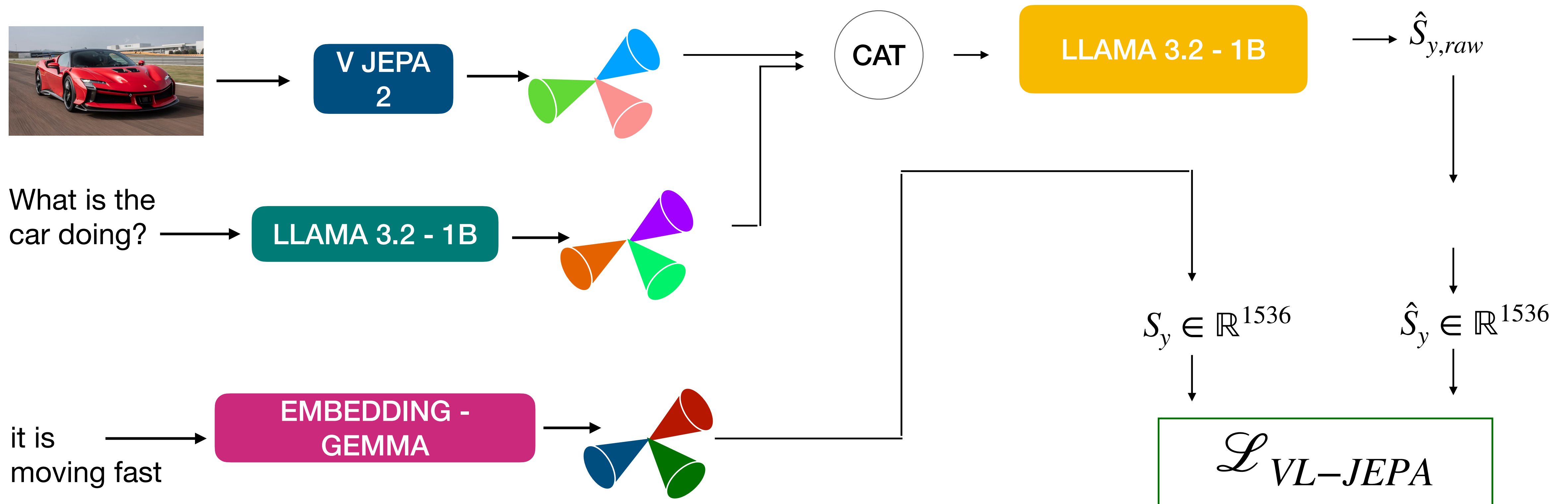
$\mathcal{L}_{VL-JEPA}$

Two Stages Training 4/4

2. Supervised Fine Tuning: by using the textual query as additional input the **VL-JEPA BASE** is now trained over Visual Question Answering task. Resulting model is named **VL-JEPA SFT**

Type of datasets: PLM data mixture: VQA, Captioning, Classification

Note: a downsampled version of pretraining data is used again in order to avoid catastrophic forgetting



4. Experiments

Experiments Classes

Several types of **experiments** have been set up to evaluate **model performances**, on different tasks.

These tasks do not only **evaluate** VL-JEPA's ability to **understand semantics** and **meanings**, but also provide a measure of how effectively the model has built a **robust representation of the world** from which the training data were provided.

Tasks:

- **Classification and Retrieval:** 8 classification datasets and 8 retrieval datasets. Compared with families of CLIP, SigLIP2, PE-Core.
- **Visual QA:** candidate answers were embedded with Y-Encoder, select the answers that minimize distance with the predicted representation. Four datasets involving: compositional QA, counting, object hallucination. Compared with BLIP and other VL families.
- **WorldPrediction-WM:** given the starting and the final state of a scenario (the world), is the model able to predict the representation of an intermediate masked state?
- **Action anticipation:** given a partial sequence of states, can the model predict the representation of future states?

Classification and Retrieval

Model		# Parameters	# Samples Seen	Zero-shot	Generalist Model	Video Classification (Top-1 Accuracy)								Text-to-video Retrieval (Recall@1)									
						Average	SSv2 [Goyal et al., 2017]	MC EK100 [Damen et al., 2022]	MC EgoExo4D [Grauman et al., 2024]	MC Kinetics-400 [Kay et al., 2017]	COIN (SR) [Tang et al., 2019]	COIN (TR) [Tang et al., 2019]	CrossTask (SR) [Zhukov et al., 2019]	CrossTask (TR) [Zhukov et al., 2019]	Average	MSR-VTT [Xu et al., 2016]	ActivityNet [Caba Heilbron et al., 2015]	DiDeMo [Anne Hendricks et al., 2017]	MSVD [Chen and Dolan, 2011]	YouCook2 [Zhou et al., 2018]	PVD-Bench [Bolya et al., 2025]	Dream-1k [Wang et al., 2024]	VDC-1k [Chai et al., 2024]
CLIP	RN50	75M	12.8B			21.8	2.1	1.5	2.1	41.4	8.6	39.0	10.9	68.7	28.3	28.7	17.7	24.7	29.7	5.1	27.6	47.2	46.0
	ViT-B	124M	12.8B	✓	✓	25.4	3.1	1.3	2.8	49.5	11.2	47.3	16.2	71.5	29.3	31.0	19.5	25.7	34.0	6.1	27.0	48.5	42.9
	ViT-L	389M	12.8B			30.7	3.8	3.7	2.6	58.3	14.7	63.5	20.8	78.5	35.3	35.9	23.4	30.7	41.9	7.9	36.7	56.8	49.3
SigLIP2	ViT-B	375M	40B			33.9	5.2	2.3	4.5	57.8	20.6	69.9	27.7	82.9	39.6	40.2	25.0	32.1	48.6	13.8	52.1	60.9	43.7
	ViT-L	882M	40B	✓	✓	38.6	5.9	4.5	6.4	63.6	24.2	78.5	35.1	90.8	45.4	41.6	32.7	35.1	53.5	19.0	59.2	71.6	50.9
	ViT-g	1.9B	40B			39.8	6.1	6.1	5.6	68.0	26.0	80.4	35.1	90.8	47.5	43.4	33.9	38.9	56.0	22.2	60.4	73.0	52.5
PE-Core	ViT-B	448M	58B			37.2	5.8	3.3	6.0	65.4	21.5	77.1	26.9	91.8	44.9	46.5	35.4	35.3	49.1	15.2	59.8	68.7	49.2
	ViT-L	671M	58B	✓	✓	42.9	9.3	6.0	11.6	73.4	27.1	83.3	37.5	95.3	50.2	48.9	41.7	40.8	56.2	22.5	64.7	75.9	51.0
	ViT-G	2.3B	86B			44.7	9.0	6.4	13.6	<u>76.4</u>	29.0	<u>86.0</u>	40.3	<u>97.2</u>	58.1	<u>51.6</u>	49.1	44.5	<u>58.7</u>	26.0	77.0	89.2	68.5
VL-JEPA _{BASE}	ViT-L	1.6B	3.3B	✓	✓	<u>52.5</u>	<u>19.3</u>	<u>21.8</u>	<u>33.2</u>	64.8	<u>47.4</u>	79.4	<u>64.5</u>	89.6	<u>63.7</u>	40.0	<u>64.9</u>	<u>50.0</u>	49.0	<u>40.4</u>	<u>83.1</u>	<u>93.3</u>	<u>88.8</u>
VL-JEPA _{SFT}	ViT-L	1.6B	3.6B	✗	✓	75.4	73.2	44.6	68.1	84.8	66.4	90.3	79.8	96.2	63.8	46.2	62.4	52.3	52.3	37.8	82.6	89.3	87.7
Previous SoTA (including specialist models)				✗	✗	-	77.5	56.4	47.8	92.1	67.3	95.3	64.5	96.0	-	62.8	74.1	74.2	61.4	28.9	77.0	89.2	68.5

Visual QA



GQA: compositional visual reasoning

Model	Accuracy
BLIP-2 (OPT-2.7B)	33.9
BLIP-2 (FlanT5XXL)	41.0
InstructBLIP (FlanT5XL)	48.4
InstructBLIP (Vicuna-13B)	49.5
Qwen-VL-Chat-7B	57.5
Qwen-VL-7B	59.3
InternVL-Chat (Vicuna-7B)	59.5
LLaVA-1.5 (Vicuna-7B)	62.0
InternVL-Chat (Vicuna-13B)	66.6
VL-JEPA_{SFT} (1.6B)	61.5

TallyQA: complex object counting

Model	Accuracy
SmolVLM-256M	32.3
SmolVLM-500M	44.8
PaLI-700M	62.3
SmolVLM-2B	64.7
PaLI-3B	65.8
InstructBLIP (Vicuna-13B)	68.0
PaLI-17B	71.9
LLaVA-1.5 (Vicuna-13B)	72.3
PaliGemma (3B)	76.8
VL-JEPA_{SFT} (1.6B)	69.9

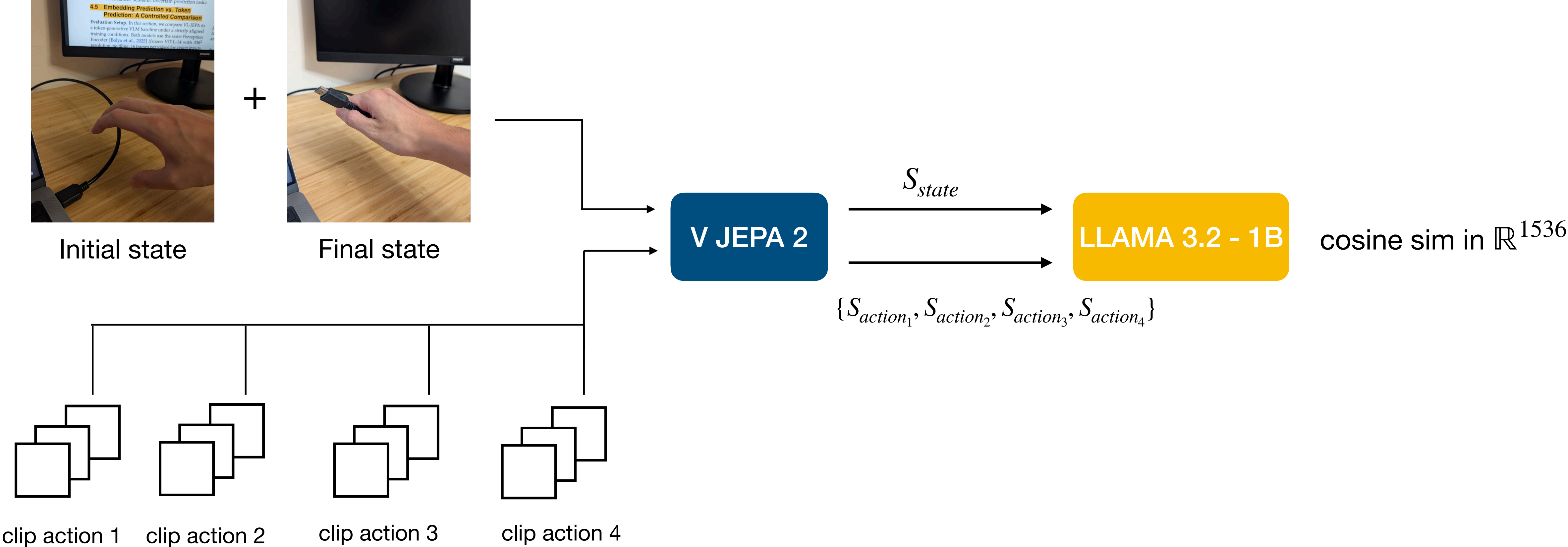
POPE: object hallucination

Model	Accuracy
SmolVLM2-256M	56.4
SmolVLM-256M	57.9
LLaVA-7B	72.9
InstructBLIP (Vicuna-13B)	79.0
Video-LLaVA (7B)	83.4
SmolVLM-500M	85.8
LLaVA-1.5-7B	85.9
LLaVA-1.5-13B-HD	86.3
SmolVLM-2B	87.5
VL-JEPA_{SFT} (1.6B)	85.7

POPEv2: object hallucination

Model	Accuracy
SmolVLM-256M	62.3
LLaVA-1.5-13B	72.7
InternVL2-8B	74.5
InternVL2-26B	76.1
Qwen2-VL-72B	79.4
SmolVLM-500M	83.8
Qwen2-VL-7B	87.0
SmolVLM-2B	88.8
Qwen2-VL-2B	91.3
VL-JEPA_{SFT} (1.6B)	86.3

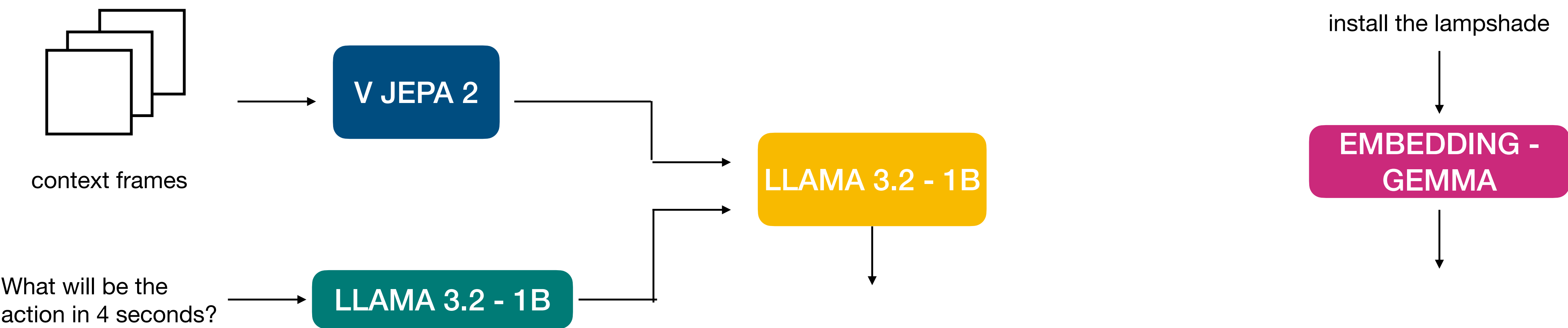
WorldPrediction-WM



Vision Language Models								Socratic LLMs (w/ Qwen2.5-VL-72B captions)									VL-JEPA		
InternVL2.5				Qwen2.5-VL				Llama-3.1		Llama-4		Qwen2.5			GPT-4o	Claude-3.5	Gemini-2	BASE	SFT
2B	4B	26B	38B	3B	7B	32B	72B	8B	70B	109B	400B	3B	7B	72B	N/A	N/A	N/A	1.6B	1.6B
20.0	29.8	30.2	50.3	21.6	45.5	49.0	57.0	48.7	49.8	52.7	53.6	44.0	49.1	48.5	52.0	53.3	55.6	63.9	<u>65.7</u>

Action Anticipation

Actions forecast in frame sequences. Datasets: COIN and EPIC-KITCHENS



Model	Vision Encoder	$t = 1s$	$t = 2s$	$t = 4s$	$t = 10s$
V-JEPA2	ViT-g-384px	<u>39.7</u>	<u>28.6</u>	19.3	8.2
V-JEPA2	ViT-L-256px	32.7	23.4	15.9	7.1
VL-JEPA	ViT-L-256px	34.2	26.0	<u>19.4</u>	<u>11.7</u>

Recall@5 - Epic Kitchens

Model	VideoLLM -online (8B)	VideoLLM -MoD (8B)	ProVideLLM -8B/11+	VL-JEPA (1.6B)
Accuracy	49.1	49.7	53.6	56.2

Accuracy - COIN

On World Prediction and Action Anticipation

(This slide wants to resume some personal reflections about WP, AA and the power of JEPA-like models)

It worth taking a moment to reflect on **WP** and **AA** tasks and how they can be interpreted in the context of how VL-JEPA works.

Both these tasks involve the model capacity to **infer something** that is actually **missing** from the original **input** (masked in WP, “unknown” in AA). We have seen that VL-JEPA operates in the **representation space** with InfoNCE loss. This implies that: if the **inferred representation** of the **missing part** is **very close** to the **representation** of the real missing “input”, the **learnt representation manifold** is very smooth.

The fact that the **abstract representation of a concept** is **close** to the **abstract representation of another**, implies that **somewhere in between those abstract representation** relies something that is very close to **both the original inputs passed to the model**.

This (at least in principle) could allow us to **plot trajectories in the latent space** probably this is the reason why JEPAs are considered great models for planning purposes (online LeCun interviews).

If we take this point of view to the extreme: if we block the training when InfoNCE is not “fully” optimized, and we make inference over a WP task, the model could predict a generic red fast moving object, instead of an italian super car.

[META's research page about V-JEPA2 \(JEPAs family\) provides very good examples of this concept](#)

5. Embedding Prediction vs Token Prediction

Controlled Comparison

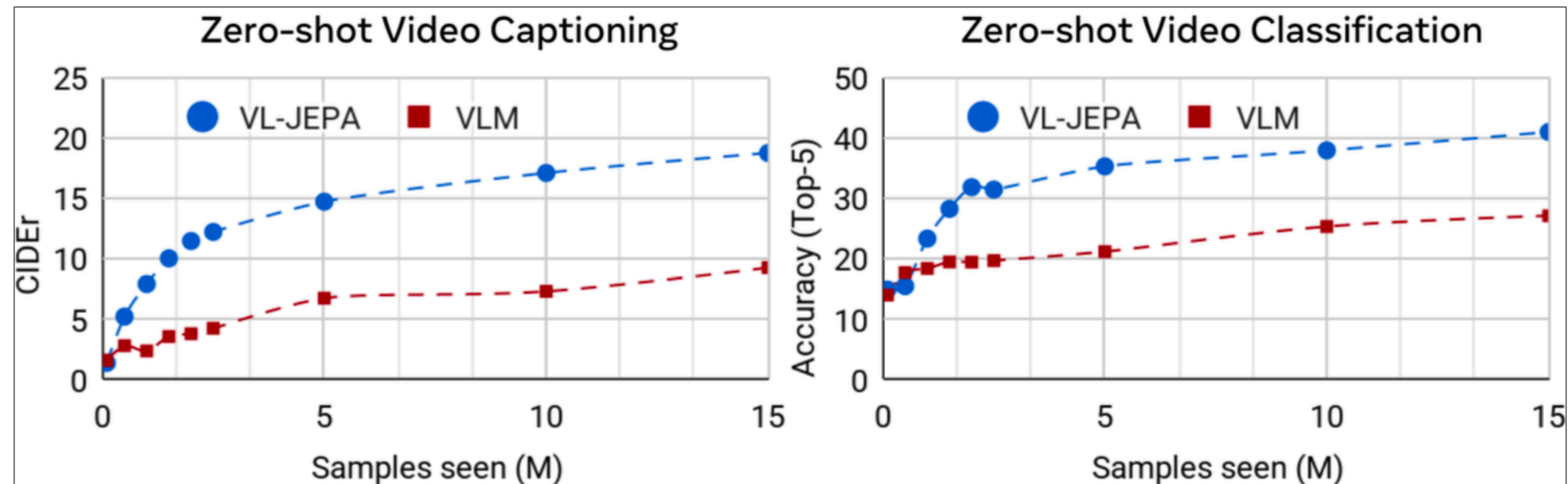
VL-JEPA has been compared with a **VLM** baseline on zero shot video captioning and zero-shot classification with following settings:

- Same **encoder** for **vision inputs**
- Same **batch size** (128)
- Same **learning rate scheduler**
- Same **training data**

How to evaluate **different types of results**?

VL-JEPA: candidate is the the embedding with lowest cosine distance “Closest Embedding Classifier”

VLM: predicted class with lower perplexity



6. Conclusions

Conclusions

The article introduced **VL-JEPA**, a **Vision Language** model defined over the **Joint Embedding Predictive Architecture**.

Like all architectures in JEPA family, it exploits **representation-space embeddings** to learn.

In VL-JEPA this concept is adapted in **aligning Visual** and **Textual semantics** in the **same representation space**, by optionally **conditioning** the **visual input** on **additional textual queries**, guaranteeing **alignment** not between exact elements, but **between complex abstractions** of them.

This level of abstraction allows the model to build a **smooth internal representation of the world**, that is provided by the data on which it is trained, enabling it to make **future states inference** (such as planning tasks).

Although this **internal representation** is arguably the most interesting characteristic of VL-JEPA, the presented experiments also shows that the model sets (on average) state of art performances on visual-language alignment tasks such as Video classification and text to video retrieval.

References: [1] VL-JEPA: Joint Embedding Predictive Architecture for Vision-language